

Reviewer Pack — Minimal Number of Noncrossing Partitions and Resolution Proo...

Entry aad5ee6cc69d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 08:29:37 UTC

Minimal Number of Noncrossing Partitions and Resolution Proof Width Correlation

Entry ID: aad5ee6cc69d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 08:29:37 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Enumeration (Noncrossing Partitions) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every satisfiability problem instance φ , the resolution proof width $w(\varphi)$ is linearly correlated with the minimal number of noncrossing partitions $n_{\text{nncp}}(\varphi)$, such that $w(\varphi) = \Theta(n_{\text{nncp}}(\varphi))$.

Rationale (proposer's reasoning):

The combinatorial nature of noncrossing partitions may provide a novel way to encode and measure the complexity of resolution proofs, potentially revealing structural properties not captured by traditional invariants.

Taxonomy category: combinatorial_enumeration_noncrossing_partitions_resolution_proof_complexity_c (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f3823e66171ec5b0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the resolution proof width and the minimal number of noncrossing partitions for all satisfiability instances exceeds 0.9, with p-value < 0.05 . The conjecture is falsified if any seed instance has a Pearson correlation coefficient ≤ 0.6 or a p-value ≥ 0.1 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncrossing partitions" AND "resolution proof complexity" - "minimal number of noncrossing partitions" AND resolution width - "correlation between resolution proof width and noncrossing partitions"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        clauses = []
        for _ in range(n):
            clause = [random.randint(1, n * 2) for _ in range(random.randint(1, 3))]
            clauses.append(clause)
        return clauses

    def is_satisfiable(instance):
        # Simplified SAT solver using backtracking
        assignment = {i: None for i in range(1, len(instance) * 2 + 1)}

        def backtrack(i):
            if i > len(instance):
                return True
            for val in [True, False]:
                assignment[instance[i-1][0]] = val
                if all(any(not (x < 0 and not assignment[-x]) for x in clause) for clause in instance[i-1]):
                    if backtrack(i + 1):
                        return True
                assignment[instance[i-1][0]] = None
            return False

        return backtrack(1)

    def resolution_width(instance):
        # Simplified resolution width calculation
        clauses = set(tuple(sorted(clause)) for clause in instance)
```

```

queue = list(clauses)
while queue:
    clause = queue.pop()
    if len(clause) == 0:
        return float('inf')
    unit_clause = next((x for x in clause if abs(x) not in assignment), None)
    if unit_clause is None:
        continue
    unit_val = assignment[unit_clause]
    new_clauses = set()
    for c in queue:
        if any(abs(x) == abs(unit_clause) and (x > 0) != unit_val for x in c):
            continue
        new_c = tuple(sorted(set(c) - {unit_clause, -unit_clause}))
        if len(new_c) == 1:
            return float('inf')
        new_clauses.add(new_c)
    queue.extend(new_clauses)
return max(len(clause) for clause in clauses)

def noncrossing_partitions(n):
    # Simplified calculation of minimal number of noncrossing partitions
    if n <= 1:
        return 1
    count = 0
    for i in range(1, n):
        count += noncrossing_partitions(i) * noncrossing_partitions(n - i)
    return count

results = []
for n in [5, 10, 15, 20, 30, 40]:
    instance = generate_instance(n)
    if is_satisfiable(instance):
        width = resolution_width(instance)
        partitions = noncrossing_partitions(n)
        results.append((width, partitions))

if not results:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No satisfiable instances generated"
    }

widths = [r[0] for r in results]
partitions = [r[1] for r in results]

n = len(widths)
mean_width = sum(widths) / n
mean_partitions = sum(partitions) / n

covariance = sum((widths[i] - mean_width) * (partitions[i] - mean_partitions) for i in range(n))
width_variance = sum((widths[i] - mean_width) ** 2 for i in range(n)) / n

```

```

partitions_variance = sum((partitions[i] - mean_partitions) ** 2 for i in range(n)) / n

correlation_coefficient = covariance / (math.sqrt(width_variance) * math.sqrt(partitions_variance))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(len(instance) for instance, _ in results),
    "conjecture_holds": correlation_coefficient > 0.9 and n >= 30,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(r["metric_value"] <= 0.6 or r["p_value"] >= 0.1 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"low correlation\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8d0d8396.py", line 123, in <module>
    trial_result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8d0d8396.py", line 81, in run_trial
    width = resolution_width(instance)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8d0d8396.py", line 53, in resolution_width
    unit_clause = next((x for x in clause if abs(x) not in assignment), None)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8d0d8396.py", line 53, in <genexpr>
    unit_clause = next((x for x in clause if abs(x) not in assignment), None)
    ~~~~~

```

NameError: name 'assignment' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the Pearson correlation coefficient and p-value could not be calculated to verify the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again to obtain valid results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15244
2	propose	ollama_remote	glm4:latest	0	0	13279
3	preregistration	ollama_remote	glm4:latest	0	0	15777
4	novelty	ollama_remote	glm4:latest	0	0	13710
5	novelty	ollama_remote	glm4:latest	0	0	13708
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14896
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14530
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12901
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21869
10	judge	ollama_remote	glm4:latest	0	0	8875

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 144788 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/aad5ee6cc69d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aad5ee6cc69d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/aad5ee6cc69d.tar.gz` (if generated)